DATABASE SECURITY / VISUAL LEARNING EDITION

SQL Injection Explained

From database basics to finding, proving, and preventing SQL injection

UNDERSTAND

Read the query

INVESTIGATE

Prove the cause

DEFEND

Verify the repair

40 illustrated chapters / 120 knowledge checks / local practice lab

FREE PREVIEW · FOUNDATIONS + A COMPLETE SQLi LESSON

HAROON SHAHID

FIRST EDITION / VERSION 1.0 / 23 OCTOBER 2026

Edition and reader information

First edition · Version 1.0 · 23 October 2026

Published by Haroon Shahid. Copyright © 2026 Haroon Shahid. All rights reserved except where the included license expressly permits use. This independent publication is not endorsed by the database vendors, OWASP, or the authors cited here.

The examples and diagrams are original teaching material. Vendor names identify the systems being discussed. Sources retain their own rights and licenses. The source catalog distinguishes implementation guidance, historical evidence, and further study.

What you are buying

This is a book, an offline reading edition, a local practice application, a guided workbook, answer explanations, and a review system. Its purpose is to build the ability to explain a query, trace a flaw, collect proportionate evidence, and verify a repair. Completion is not a professional certification or a guarantee of expert performance.

The shared training environment is Northstar Books, a fictional shop. All names, passcodes, invoices, tokens, and PINs in the lab are dummy data. Run the vulnerable application only on your own computer at the supplied loopback address. Test other systems only with explicit permission and an agreed scope. An exposed public page is not permission to probe it.

How this edition was checked

Core SQL and application exercises were executed against the included SQLite lab. The build's validation report names the tested runtime, engine, browser, and checks. Examples for other database servers and frameworks are documented integration patterns, not a claim that those systems were installed and tested for this edition. Always check your exact driver, version, configuration, and permissions.

The lab uses Node.js 24.12 or later in the 24 series, or a later compatible supported release. It needs no npm packages. SQLite is embedded in Node.js. The short delay function and separate signal board are explicitly supplied by the lab; they are not native SQLite network or sleep features. The signal board sends no DNS or HTTP callback.

Editorial transparency and updates

This publication was developed with AI assistance for research synthesis, drafting, code, diagrams, and production. Program execution and source checks reduce errors; they do not make errors impossible. The dated source catalog lets readers inspect important claims. Future database and framework releases may change interfaces.

For a correction, reply to your Gumroad receipt email. Include the edition, chapter, page or section, expected behavior, and a small dummy-data reproduction. Do not send real credentials, customer records, or private database exports. This purchase includes this edition and its supplied files; it makes no promise of lifetime updates or personal consulting.

Preview: Chapters 1–3 teach foundations. Chapter 15 shows a complete SQLi lesson and normally follows Chapters 4–14. Answers are included. The practice app is in the full package.

How to use this book

Begin with one question

You type a category into a bookshop search box. The shop should show books in that category. Why could that simple action ever reveal a private row?

The answer is not that apostrophes are evil. It is that two different things were mixed together: a program that describes what the database should do, and a value that the user should be allowed to choose. SQL injection happens when that value gains the power to change the program.

You will learn enough SQL to see the change yourself. Then you will follow one fictional shop's requests through application code, a driver, the engine, and the visible result.

What you need before starting

No SQL or programming experience is needed. The companion start page explains setup one step at a time. If SQL is new, read Chapters 1–10 before attempting the injection exercises.

We define each new term when you need it. The glossary gives a second route back to the main ideas.

The rhythm of a lesson

Read the objective, figure, and explanation. Predict the example's outcome, then run it. Explain the result in your own words. Answer the three checks before reading their explanations. State what the repair must preserve and stop.

Figures explain the relationship named in their captions. Vendor chapters add engine-specific exceptions.

Five questions to carry through the whole book

What can the caller control? Where does that value go? Which language interprets it? What authority does that operation have? Which observation proves the claim?

Those questions prevent false conclusions. An error can be a clue without proving disclosure. A bound query can still lack authorization. One slow response does not establish an injected delay.

Use both paths

Every application exercise has an intentionally vulnerable path and a safer path, except the SQL workbench, where you deliberately write SQL. First understand the ordinary feature. Then demonstrate the flaw using only the fictional data. Finally run the same input against the repair and check the ordinary feature again.

The goal is not a collection of strings that work somewhere. It is the ability to derive a small proof from the actual query, explain its limits, and fix the boundary that failed.

Learning roadmap

Stage	Chapters	Do before moving on
Read the database	1–10	Predict results for filters, NULL, joins, aggregates, UNION, and a rollback.
Trace the application	11–14	Draw a complete request-to-query path and name the web and database identities.
Find and prove a flaw	15–20	Produce a controlled local result change, a dummy UNION marker, and a reliable blind signal.
Explain difficult paths	21–25	Trace a delayed consumer and explain grammar, transport, and execution limits.
Compare real platforms	26–30	Use the correct engine manual instead of assuming SQLite behavior transfers unchanged.
Build and verify defenses	31–36	Bind values, control structure, enforce authorization, and preserve feature behavior in tests.
Work independently	37–40	Complete the capstone and write a finding whose evidence supports every claim.

Choose a route without losing the foundations

A first-time learner should read in order. A developer can first read Chapters 11–15 and 31–36, then return to any SQL foundation they cannot explain. A security learner with SQL experience should start with the baseline and control discipline in Chapter 16, then complete every repaired comparison. A teacher can use each part as a lesson unit and the capstone as an assessment.

A sensible session is one chapter, one experiment, and one explanation. Stop earlier if you cannot predict the next query. Reading quickly is not a substitute for correcting a wrong model.

Spaced review that measures understanding

At the end of a session, close the book and draw the main figure from memory. At the next session, answer yesterday's questions before reading new material. About a week later, repeat one experiment with a changed but ordinary input. After the capstone, review a different local query without the book open. These intervals are a practical study suggestion, not a scientific promise that one schedule fits every reader.

Keep an error notebook: my prediction; the observation; the rule I misunderstood; a new test that distinguishes the rules.

Contents

Edition and reader information 2
How to use this book 3
Learning roadmap 4
Contents 5
01 Data, databases, and the problem they solve 6
02 Tables, keys, relationships, and clean structure 8
03 Your first SELECT, read one clause at a time 10
15 Where SQL injection begins 12
Appendix A Answer explanations 14
Sources for this preview 15
Continue with the full edition 16

I READ THE DATABASE

01 Data, databases, and the problem they solve

OBJECTIVE Explain why an application uses a database and distinguish the engine from the data.

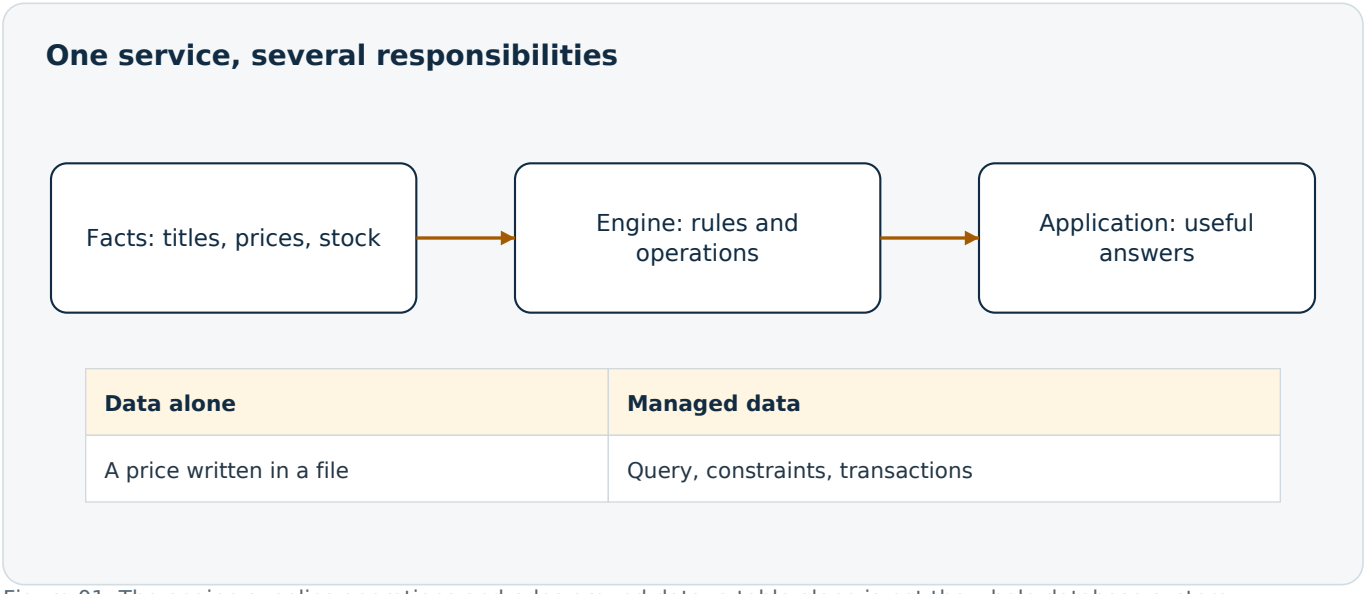


Figure 01. The engine supplies operations and rules around data; a table alone is not the whole database system.

Start with a small bookshop

Northstar Books is a fictional shop used throughout this book. It needs a catalog, stock counts, and invoices. A piece of information, such as a title or price, is data. A database organizes related data so a program can store, find, and change it under defined rules.

A spreadsheet or text file can hold a catalog. The harder questions appear when two people buy the last copy, a title changes, or an invoice must remain linked to the correct customer. Database systems provide query languages, integrity rules, transactions, and access controls to help manage these problems. They do not automatically supply the shop's business rules.

The database engine is the software that performs database operations. The database is the organized data and its definitions. A database server receives requests from other programs. SQLite is an embedded engine: the application uses a library rather than contacting a separate database service. PostgreSQL and MySQL commonly run as separate servers. The same word database is often used loosely for all three ideas. [R01]

Read the first data model

Our catalog starts with these fields:

id	title	category	price_cents
1	Web Paths	web	2500

id	title	category	price_cents
2	SQL Steps	database	3000

The amount 2500 means 25.00 units of the shop's fictional currency. Integer cents make our examples exact; real money systems must also define currency, rounding, and valid ranges. The id identifies a book. The title is a description and may change.

Think of an engine as a librarian and the stored records as the collection. A librarian can locate and update records. The librarian still needs instructions about who may see a private invoice. This analogy does not mean every database behaves like a physical filing cabinet.

Meet the learning environment

The included lab runs on your computer at http://127.0.0.1:8766. It contains only fictional records. Its SQL workbench lets you write queries in a separate practice database. Later application exercises demonstrate mistakes in the code that builds queries. An intended SQL console is not, by itself, a SQL injection vulnerability.

Open the workbench, run the query below, and identify the two displayed fields:

```
SELECT id, title FROM books ORDER BY id;
```

You will learn every word in Chapter 3. For now, notice that a program asks a structured question and receives structured rows.

Remember: A database can enforce declared rules. It cannot guess the security policy the application forgot to express.

KEY TAKEAWAY
Separate the stored facts, the engine, and the application rules.

Check your understanding

1.1 Is a database engine the same as the stored records?

1.2 Why use an id when a book already has a title?

1.3 Does a working SQL console prove SQL injection?

Reflection Which two people or processes might try to change the same record in a familiar service?

Answer explanations in Appendix A | Sources: R01, R03

I READ THE DATABASE

02 Tables, keys, relationships, and clean structure

OBJECTIVE Read a small relational schema and explain primary keys, foreign keys, and normalization.

A relationship is stored as a reference

authors	
id (PK)	name
1	Amina Shah
2	Bilal Khan

1 many

books	
id (PK)	author_id (FK)
1	1
3	1
5	NULL

Book 5 may have no assigned author. A present foreign key must reference a valid author.

Figure 02. One author can relate to many books. A NULL author_id represents an unassigned author in this seed, not a broken key.

Rows describe instances; columns describe fields

A table groups records with defined columns. A row is one record, such as one book. A column is a field, such as title. A schema describes database objects and rules. A value has a type or storage representation: text, integer, date, and so on. Exact type behavior differs by engine. [R02]

Northstar stores authors separately from books:

```
CREATE TABLE authors (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL
);
CREATE TABLE books_demo (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL,
  author_id INTEGER REFERENCES authors(id)
);
```

This is a teaching definition, not a command to run over the already seeded authors table. A primary key uniquely identifies a row. A foreign key links a value to an allowed parent key. NOT NULL requires a value. A UNIQUE constraint prevents duplicate values under that database's rules. A CHECK constraint expresses a condition, such as stock being non-negative.

A relationship is not a duplicated description

If two books have the same author, books.author_id can hold the same authors.id. This is a one-to-many relationship. A book with several authors needs a linking table containing book_id and author_id: a many-to-many relationship. Each association becomes its own row.

Normalization organizes facts to reduce avoidable duplication and update mistakes. Store an author's address once when it is one shared fact, rather than copying it into every book row. This is not a rule to split every string into another table. Performance, history, and product requirements affect the final design.

A useful analogy is an address book. Several letters can refer to one contact entry. Updating the entry is easier than correcting many copied addresses. Historical invoice addresses may deliberately be snapshots, so blindly replacing every copy can also be wrong.

Integrity is different from permission

A foreign key can prevent a book from pointing to a missing author. It does not decide whether a particular web user may view that author's private details. Integrity concerns valid relationships and values. Authorization concerns permitted actions. Both are necessary.

SQLite foreign-key enforcement must be enabled for the relevant connection. Our lab does that during initialization. A declaration that is not enforced is weaker than its appearance suggests. [R03] [R04]

Practice: Inspect the supplied seed.sql. Find the primary key on books, its author reference, and its stock constraint. Compare those declared rules with your drawing. You will learn how to query the engine's schema metadata in Chapter 26.

Remember: A well-designed relationship reduces mistakes, but no table structure makes unsafe application queries safe.

KEY TAKEAWAY
Keys identify and connect records; authorization controls who may use them.

Check your understanding

2.1 What does a foreign key connect?

2.2 Why can repeating an author address cause trouble?

2.3 Does a valid invoice foreign key prove the viewer may read the invoice?

Reflection Which facts should remain historical snapshots rather than follow a later profile update?

Answer explanations in Appendix A | Sources: R02, R03, R04

I READ THE DATABASE

03 Your first SELECT, read one clause at a time

OBJECTIVE Read SELECT, FROM, aliases, expressions, and the difference between SQL text and its result.

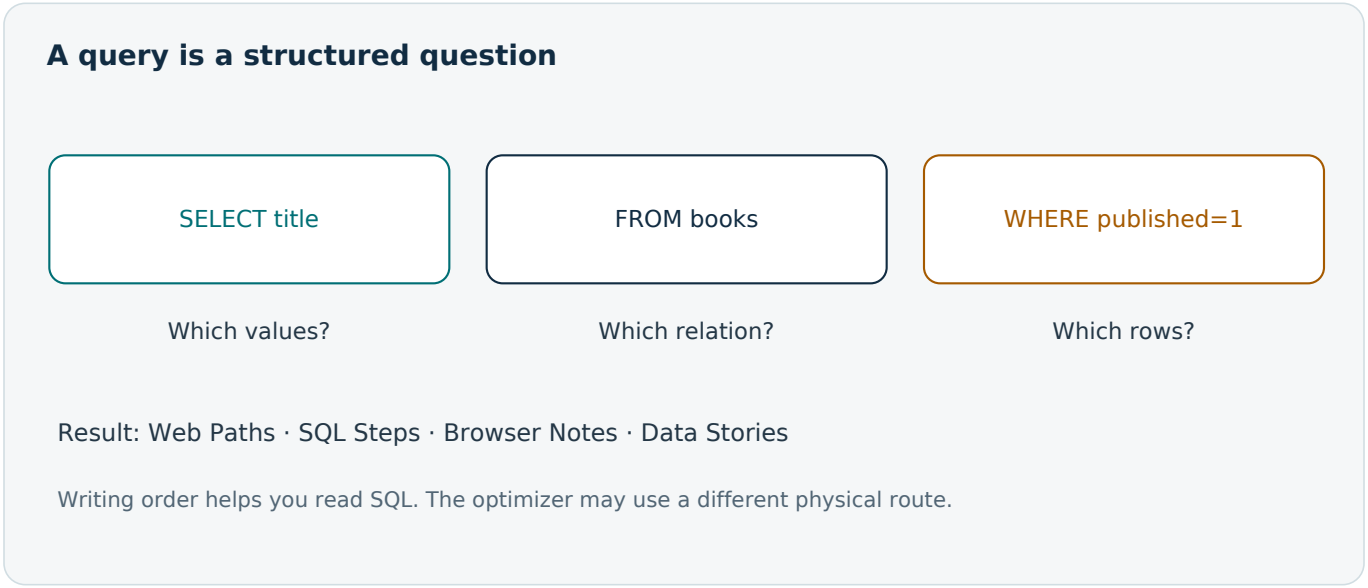


Figure 03. Read SELECT as requested columns, FROM as the input, and WHERE as a qualifying condition. This is a reading map, not a physical execution plan.

SQL asks the engine to do work

SQL stands for Structured Query Language. Many relational database systems use it to define objects, read data, change records, and control transactions. A query commonly means a request that reads data, though people sometimes use the word for any SQL statement. A statement is one complete instruction.

The same central ideas appear across products, but SQL is an ecosystem of dialects. A dialect is an engine's particular syntax and behavior. Do not assume a PostgreSQL command works unchanged in Oracle or SQLite. We use SQLite for executable foundations, then compare other engines explicitly. [R05] [R22]

Read this query from left to right before running it:

```
SELECT id, title
FROM books
WHERE published = 1
ORDER BY id;
```

SELECT names result expressions. FROM names the input table. WHERE keeps rows whose condition is true. ORDER BY defines the displayed ordering. The semicolon ends a statement in many tools; it is not a requirement to inject additional statements.

Results are not the original table

The seeded database has five books. Four have published=1, so this query returns four rows. It does not delete the draft. It creates a result containing the selected fields from qualifying rows. A result can also include computed values:

```
SELECT title, price_cents / 100.0 AS price
FROM books
WHERE id = 1;
```

The output is Web Paths and 25.0. AS gives the expression an alias, a result label. The decimal 100.0 avoids an integer-division lesson being hidden inside a price example. Numeric conversion and decimal precision need separate review in real financial code.

A string literal is text written inside SQL quotes: 'web'. The unquoted name title refers to a column in this context. SQL keywords are instructions such as SELECT. A value, an identifier, and a keyword have different grammatical jobs. That difference becomes central to parameter binding.

Think of a restaurant order: choose these dishes from this menu, subject to these conditions. Asking for a different dish is different from changing the restaurant's ordering rules. A query is declarative: it describes the result, while the engine chooses much of the physical work.

Practice: Select only title and stock for book 2. Expected: SQL Steps and 2. Then remove the WHERE clause and explain why more rows appear. Use explicit columns while learning; SELECT * is convenient but can accidentally expose newly added fields.

Remember: Explain what a clause means before memorizing its spelling. The result is evidence of the engine's interpretation.

KEY TAKEAWAY
A SELECT describes a result; it does not grant the caller permission to rewrite its structure.

Check your understanding

3.1 What does AS change?

3.2 Does selecting published books delete the draft?

3.3 Why distinguish title from a quoted text literal?

Reflection Explain one query aloud without using the words just or obviously.

Answer explanations in Appendix A | Sources: R05, R06, R22

III FIND AND PROVE THE FLAW

15 Where SQL injection begins

OBJECTIVE Explain injection as a change of query structure and demonstrate the first local proof.

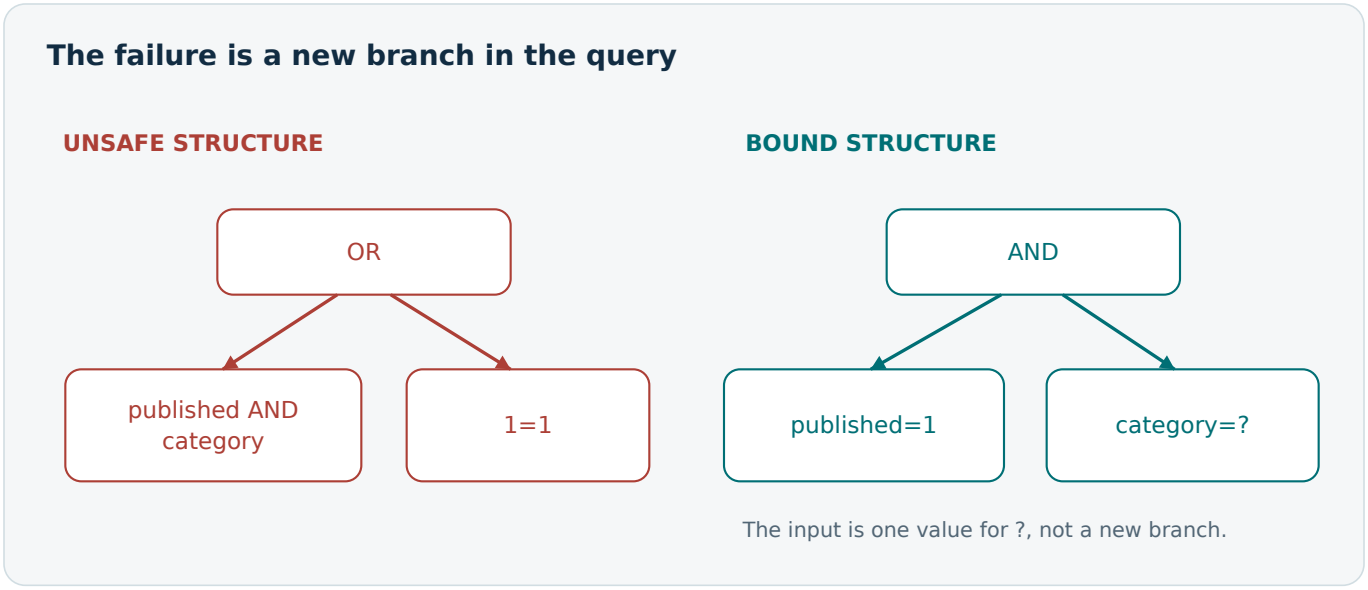


Figure 15. The vulnerable input creates an OR branch. In the bound query, the whole string occupies one value position.

You now know enough SQL to recognize the failure

SQL injection happens when untrusted input changes the structure or meaning of an SQL command the application executes. The caller should be able to choose an ordinary value, such as a category. The vulnerable design lets that caller supply part of the database program. [R15] [R17]

The local catalog builds this unsafe statement:

```
const sql = "SELECT id,title,category FROM books " +
  "WHERE published=1 AND category='" + value + "'";
const result = db.prepare(sql).all();
```

The method name prepare does not rescue it. The application has already inserted the value into SQL text before preparing. The engine prepares whatever program the application supplied, including any attacker-created structure.

Follow the complete local example

The baseline value web returns two published web books. Now enter this in Visible catalog's vulnerable path:

```
' OR 1=1 --
```

The completed condition is equivalent to published=1 AND category='' OR 1=1, with the trailing quote commented out. Because AND binds more tightly than OR, the true alternative also permits the unpublished Draft Manual. Our integration test confirms that fictional record appears. This is an authorization-relevant change caused through the application's category input.

The goal is not the magic spelling OR 1=1. It is to explain the original context, the quote boundary, the new operator, the remaining syntax, the engine's logic, and the resulting rows. Another query shape may require a different proof.

Think of a library request card. A reader should write a category in its blank space. The vulnerable application copies that writing into the librarian's instruction manual, so it can become a new rule about which shelves to open.

Repair the actual boundary

The safe path keeps the statement structure fixed:

```
const sql = "SELECT id,title,category FROM books " +
  "WHERE published=1 AND category=?";
const result = db.prepare(sql).all(value);
```

The entire injected string is now one category value. It matches no category and cannot add an OR branch. Repeat the exact same input and confirm zero rows. Then test web again and confirm normal behavior still works.

Know what you have and have not proved

This proof establishes SQL structure control and exposure of a seeded unpublished row in this local path. It does not prove operating-system command execution, every table being readable, or compromise of another service. Privileges, query context, driver behavior, and reachable functions determine further impact.

Practice: Write a five-line trace from JSON value to generated SQL, returned record, fixed query, and fixed result. Keep both the ordinary and adversarial tests.

Remember: A safe-looking API called with an already unsafe SQL string is still unsafe.

KEY TAKEAWAY
Keep the SQL program fixed and supply untrusted values through the binding interface.

Check your understanding

15.1 Why is prepare(sql).all() vulnerable in the first example?

15.2 What local evidence shows more than a syntax error?

15.3 Why retest ordinary web after the fix?

Reflection Can you explain each character of the local proof without calling it a magic payload?

Answer explanations in Appendix A | Sources: R13, R15, R17

Appendix A Answer explanations

Answer before reading the explanation. If your prediction was wrong, state the rule you will use next time.

01 Data, databases, and the problem they solve

1.1 No. The engine processes operations; the records are part of the database it manages.

1.2 The title may change or repeat. An identifier gives the application a stable reference.

1.3 No. A console intentionally accepts SQL. Injection requires crossing a boundary where the caller should supply data rather than query instructions.

02 Tables, keys, relationships, and clean structure

2.1 A value in one table to an allowed key in a referenced table.

2.2 Different copies may be updated inconsistently, leaving conflicting facts.

2.3 No. The application still needs an authorization decision.

03 Your first SELECT, read one clause at a time

3.1 The label of an output expression, not the stored column definition.

3.2 No. It filters the result of this query.

3.3 One names a column in this context; the other supplies a value. They occupy different grammar positions.

15 Where SQL injection begins

15.1 Untrusted text was concatenated into sql before preparation. The engine prepares the altered command.

15.2 The category input changes the result to include the unpublished seeded book, and the generated query explains why.

15.3 A useful repair must preserve the intended feature as well as stop the injected input.

Sources for this preview

Curated public source catalog, reviewed for this edition on 23 October 2026. Core guidance is from engine vendors, driver authors, OWASP, and primary historical records. The catalog is not an exhaustive systematic review. A cited source supports the nearby mechanism; it does not endorse this book. Current-version URLs can change. Explicitly versioned manuals are labeled. Historical research is a bridge for further study, with dated assumptions.

R01 SQLite: an appropriate use for a database engine

Engine guidance · Embedded storage, server databases, and architectural differences.

Open source: www.sqlite.org

R02 SQLite: data types

Engine reference · Storage classes, affinity, comparisons, and STRICT-table context.

Open source: www.sqlite.org

R03 SQLite: CREATE TABLE

Language reference · Keys, defaults, checks, and column definitions.

Open source: www.sqlite.org

R04 SQLite: foreign keys

Engine reference · Enable enforcement per connection; referential integrity is not user authorization.

Open source: www.sqlite.org

R05 SQLite: SELECT

Language reference · Joins, aggregation, compound queries, ordering, and limits.

Open source: www.sqlite.org

R06 SQLite: expressions

Language reference · Parameters, operators, NULL, CASE, and subqueries.

Open source: www.sqlite.org

R13 Node.js 24: SQLite API

Versioned runtime reference · DatabaseSync, binding, authorizers, functions, and defensive configuration; core lab requires 24.12 or later.

Open source: nodejs.org

R15 OWASP: SQL Injection Prevention

Security guidance · Binding, structural allowlists, procedure review, and least privilege.

Open source: cheatsheetseries.owasp.org

R16 OWASP: Query Parameterization

Implementation guidance · Compare API-specific bind conventions rather than copying placeholder spelling.

Open source: cheatsheetseries.owasp.org

R17 PortSwigger: SQL injection

Authorized learning path · Mechanism, impact, query contexts, second-order behavior, and free labs.

Open source: portswigger.net

R20 PortSwigger: blind SQL injection

Educational reference · Conditional responses, errors, timing, and separate channels.

Open source: portswigger.net

R22 PostgreSQL: tutorial

Vendor tutorial · Foundational queries, joins, aggregates, and transactions.

Open source: www.postgresql.org

R44 node-postgres: queries

Driver documentation · Fixed text plus values, identifiers, and named statement caching.

Open source: node-postgres.com

Continue with the full edition

The complete edition continues through application architecture, controlled discovery and exploitation, difficult query contexts, six database platforms, defensive implementation, and independent review. It includes the local Northstar Books lab and guided workbook. This preview contains Chapters 1–3, Chapter 15, and their answers. Follow the full sequence to build the prerequisites between them.

Published by Haroon Shahid. Use the seller's product page to inspect the current full-edition files and terms before buying.